

Desafio V360 – Conector de Pedidos de Compra

Este desafio simula, de forma simplificada, o problema que o nosso time resolve todos os dias: **conectar a plataforma V360 aos sistemas corporativos dos nossos clientes, cada um diferente do outro, entregando para a plataforma um contrato único e confiável**. Você não precisa saber nada de SAP para resolvê-lo. Todos os nomes de empresas e dados abaixo são fictícios.

O contexto de negócio

Quando uma empresa compra de um fornecedor, ela primeiro emite um **pedido de compra**: um documento que registra o que foi combinado, item por item, com material, quantidade e preço acordado. Depois, conforme a mercadoria é entregue, o fornecedor emite as **notas fiscais** cobrando o que entregou.

Antes de pagar, alguém precisa conferir se a nota corresponde ao que foi combinado no pedido: o material é o mesmo? A quantidade cabe no que ainda falta receber daquele pedido? O preço é o acordado? Na maioria das empresas, isso é feito à mão, nota por nota, por uma pessoa do time de contas a pagar. É lento, caro e passa erro.

É esse trabalho que a V360 elimina. Para automatizar a conferência, a plataforma V360 precisa consultar, no sistema do cliente, o pedido de compra correspondente à nota que acabou de chegar.

Aqui está o problema: cada cliente tem um sistema diferente, em uma versão diferente, e cada um expõe esses dados de um jeito diferente. A plataforma V360 não pode conhecer o formato de cada cliente. Se conhecesse, cada contrato novo viraria uma alteração no produto, e uma mudança no sistema de um cliente quebraria a plataforma para todos os outros.

É aí que entra o time de Engenharia de Integrações, onde você acabou de entrar. Nossa missão é construir a camada que fica no meio: ela busca os dados no formato de cada cliente e entrega para a plataforma V360 um contrato único, igual para todos. Quando um cliente novo é fechado, quem absorve a diferença é a nossa camada, não o produto.

Sua missão

Construir essa camada: um serviço backend com uma API REST que ingere os dados de pedidos de compra dos clientes, normaliza tudo para um modelo único (que você vai desenhar) e atende ao que a plataforma V360 precisa:

1. **Consultar os pedidos de compra de todos os clientes em um formato único**, com filtros úteis para quem opera: por cliente de origem, por fornecedor, por situação do pedido e apenas os que ainda têm algo a

receber. Ver o detalhe de um pedido específico, com o que já foi recebido e o que ainda falta em cada item. Um cliente de porte médio tem dezenas de milhares de pedidos em aberto, e a plataforma varre esse conjunto em lote, de madrugada, enquanto novas cargas dos clientes continuam entrando.

2. **Conferir uma nota fiscal contra um pedido de compra.** A plataforma envia os dados da nota que chegou (fornecedor e, para cada item, material, quantidade e valor total) e o seu serviço responde se ela está conforme o pedido. Se não estiver, precisa dizer **exatamente o que não bate**, de forma que a plataforma consiga mostrar isso para o usuário sem adivinhar nada.
3. **Um relatório das conferências feitas**, para o time entender o que está travando: quantas notas passaram, quantas travaram e por quais motivos.

Como esses requisitos viram endpoints, quais campos o modelo único tem, quais tipos de divergência existem e como as respostas são estruturadas são **decisões suas** e fazem parte da avaliação. A forma de ingestão dos dados também é livre: endpoint de upload, script de carga, leitura no start da aplicação, o que fizer sentido para você.

Uma dica de quem faz isso todo dia: as regras da conferência não estão escritas em nenhum lugar como especificação técnica. Elas saem do entendimento do negócio, exatamente como no enunciado acima. Se você tiver dúvida sobre uma regra, decida, deixe registrado no README por que decidiu assim e esteja pronto para defender.

Duas exigências de quem já rodou isso em produção

Diferente de um exercício, este serviço é a peça que fica entre a plataforma e o sistema do cliente, ligada o tempo todo. Duas coisas, por isso, **não são opcionais neste desafio**:

1. Os dados têm que viver em um banco de dados.

- Qual banco é escolha sua, relacional ou não, e a escolha faz parte da avaliação.
- Tanto os pedidos normalizados quanto o histórico de conferências precisam sobreviver a uma parada do serviço: subir de novo não pode perder o que já foi carregado nem o que já foi conferido.
- A modelagem é sua: como o pedido e seus itens são guardados, o que identifica um pedido de forma única (repare que o mesmo número de pedido pode existir em clientes diferentes) e quais índices sustentam as consultas do requisito 1.
- Decida e registre no README **o que acontece quando um cliente reenvia um pedido que mudou** — por exemplo, a quantidade já recebida de um item aumentou desde a última carga.
- Para conseguirmos rodar seu projeto sem instalar banco na nossa máquina, **ele precisa subir com um comando**: um `docker compose up` (ou equivalente) que levante o serviço e o banco juntos.

2. Toda interface que devolve lista tem que ser paginada.

- Vale para a consulta de pedidos e também para o relatório de conferências.
- A estratégia é sua — página e tamanho, deslocamento e limite, cursor — e você defende a escolha.
- A resposta precisa entregar à plataforma o que ela necessita para navegar sem adivinhar nada: onde ela está, se ainda existe coisa para buscar e como pedir o próximo pedaço.
- Paginação tem que conviver com os filtros do requisito 1. Filtrar e paginar ao mesmo tempo é o uso normal, não a exceção.
- Defina um tamanho de página padrão e um teto máximo. A plataforma é um cliente educado, mas você não quer descobrir em produção o que acontece quando alguém pede um milhão de registros de uma vez.

Parte 1 — Os dois primeiros clientes

Cliente Alfa Energia expõe os pedidos em JSON, com os itens aninhados dentro do pedido:

```
JSON
{
  "purchase_orders": [
    {
      "po_number": "4500001234",
      "created_at": "2026-08-05",
      "status": "open",
      "currency": "BRL",
      "vendor": { "tax_id": "23456789000101", "name": "Metalúrgica São Jorge S.A." },
      "items": [
        {
          "line": 10,
          "material": "MAT-1001",
          "description": "Chapa de aço 2mm",
          "uom": "UN",
          "quantity_ordered": 100,
          "quantity_received": 60,
          "unit_price": 45.9
        },
        {
          "line": 20,
          "material": "MAT-1002",
          "description": "Perfil U 3m",
          "uom": "UN",
          "quantity_ordered": 50,
          "quantity_received": 0,
          "unit_price": 128.75
        }
      ]
    }
  ]
}
```

```
}
]
}
]
}
```

O campo **status** do pedido pode ser **open**, **closed** (já encerrado) ou **blocked** (bloqueado pelo cliente).

Cliente Beta Alimentos só consegue exportar CSV separado por ponto e vírgula, no padrão brasileiro, e em **dois arquivos**: um com os cabeçalhos dos pedidos e outro com os itens. A ligação entre eles é o número do pedido.

cabecalho.csv:

```
None
NUMERO_PEDIDO;FORNECEDOR_CNPJ;FORNECEDOR_RAZAO_SOCIAL;EMISSAO;SITUACAO;MOEDA
20260088412;12.345.678/0001-90;Distribuidora Horizonte Ltda;15/08/2026;EM
ABERTO;BRL
20260088413;98.765.432/0001-55;Frigorífico Boa Mesa
S.A.;01/08/2026;BLOQUEADO;BRL
```

itens.csv:

```
None
NUMERO_PEDIDO;ITEM;CODIGO_MATERIAL;DESCRICAO;UNIDADE;QTD_PEDIDA;QTD_RECEBIDA;PR
ECO_UNITARIO
20260088412;1;MAT-77;Óleo de soja 900ml;UN;1.200,000;400,000;6,49
20260088412;2;MAT-78;Açúcar refinado 1kg;UN;500,000;0,000;4,15
20260088413;1;MAT-91;Carne bovina dianteiro kg;KG;2.000,000;0,000;27,90
```

Repare que os dois clientes falam do mesmo conceito (um pedido de compra, com fornecedor, situação e itens com quantidade e preço), mas discordam em quase tudo: estrutura dos arquivos, nomes de campos, idioma, formato de data, formato de número, máscara de CNPJ e vocabulário de situação. Faz parte do desafio decidir como unificar isso.

Os exemplos acima são amostras: seu serviço deve funcionar para qualquer volume de pedidos nesses formatos.

Ao concluir a Parte 1, faça um commit (ou tag) marcando esse ponto, por exemplo **git tag parte-1**. Isso é importante para a avaliação da Parte 2.

Parte 2 — Chegaram dois clientes novos

Na vida real, o requisito muda depois que o código está pronto. Aqui também: a empresa fechou dois contratos de uma vez, e os dois clientes entram agora, com a Parte 1 já fechada.

Cliente Gama Logística tem o sistema mais antigo da carteira e exporta tudo achatado, uma linha por item de pedido:

JSON

```
[
  {
    "ped": "GL-778",
    "item": 1,
    "cnpj_fornecedor": "34567890000112",
    "nome_fornecedor": "Transportes Ideal ME",
    "dt_criacao": 1786752000,
    "cod_mat": "TRP-01",
    "desc_mat": "Pallet de madeira",
    "um": "CX",
    "fator_conv": 12,
    "qtd_ped": 10,
    "qtd_rec": 2,
    "preco_unit_centavos": 120000,
    "situacao": 1
  },
  {
    "ped": "GL-778",
    "item": 2,
    "cnpj_fornecedor": "34567890000112",
    "nome_fornecedor": "Transportes Ideal ME",
    "dt_criacao": 1786752000,
    "cod_mat": "TRP-09",
    "desc_mat": "Caixa organizadora",
    "um": "CX",
    "fator_conv": 3,
    "qtd_ped": 4,
    "qtd_rec": 0,
    "preco_unit_centavos": 10000,
    "situacao": 1
  },
  {
    "ped": "GL-779",
    "item": 1,
    "cnpj_fornecedor": "56789012000134",
    "nome_fornecedor": "Armazéns Rio Claro Ltda",
    "dt_criacao": 1784160000,
    "cod_mat": "ARM-10",
    "desc_mat": "Estrado metálico",
    "um": "UN",
    "fator_conv": 1,
  }
```

```

    "qtd_ped": 100,
    "qtd_rec": 100,
    "preco_unit_centavos": 3500,
    "situacao": 2
  }
]

```

Particularidades do Gama:

- Não existe cabeçalho de pedido: os dados do pedido se repetem em cada linha de item.
- As datas vêm como timestamp Unix (em segundos).
- Os valores vêm em **centavos**.
- A situação é um código numérico: **1** = em aberto, **2** = encerrado, **3** = bloqueado.
- A quantidade e o preço estão na **unidade de compra**, que pode ser caixa (**CX**), com o fator de conversão informado em **fator_conv**. No exemplo, o item 1 do pedido **GL-778** são 10 caixas de 12 unidades a R\$ 1.200,00 a caixa. **As notas fiscais dos fornecedores sempre informam quantidade em unidades**, nunca em caixas.

Cliente Delta Distribuição usa exatamente os mesmos nomes de campo do Alfa Energia — é o mesmo produto de mercado, em outra versão. A diferença está em como os dados saem: o sistema do Delta **não devolve o pedido com os itens dentro**. Ele expõe duas consultas separadas, uma de pedidos e outra de itens, e a ligação entre as duas é o campo **purchase_order**, que vem em cada item.

Consulta de pedidos (cabeçalho, sem itens):

```

JSON
{
  "orders": [
    {
      "po_number": "DL-2026-0044",
      "created_at": "2026-09-02",
      "status": "open",
      "currency": "BRL",
      "vendor": { "tax_id": "67890123000145", "name": "Embalagens Norte Sul
Ltda" }
    },
    {
      "po_number": "DL-2026-0045",
      "created_at": "2026-08-28",
      "status": "closed",
      "currency": "BRL",

```

```

        "vendor": { "tax_id": "78901234000156", "name": "Papeleria Central S.A."
    },
    {
        "po_number": "DL-2026-0046",
        "created_at": "2026-09-10",
        "status": "open",
        "currency": "BRL",
        "vendor": { "tax_id": "67890123000145", "name": "Embalagens Norte Sul
Ltda" }
    }
]
}

```

Consulta de itens (de todos os pedidos, cada item apontando para o seu):

```

JSON
{
  "items": [
    {
      "purchase_order": "DL-2026-0044",
      "created_at": "2026-09-02",
      "line": 10,
      "material": "EMB-500",
      "description": "Caixa papelão 40x30",
      "uom": "UN",
      "quantity_ordered": 800,
      "quantity_received": 300,
      "unit_price": 3.75
    },
    {
      "purchase_order": "DL-2026-0044",
      "created_at": "2026-09-08",
      "line": 20,
      "material": "EMB-720",
      "description": "Fita adesiva 48mm",
      "uom": "UN",
      "quantity_ordered": 240,
      "quantity_received": 0,
      "unit_price": 8.2
    },
    {
      "purchase_order": "DL-2026-0045",
      "created_at": "2026-08-28",
      "line": 10,
      "material": "PAP-100",
      "description": "Papel A4 resma 500 folhas",
      "uom": "UN",

```

```

    "quantity_ordered": 150,
    "quantity_received": 150,
    "unit_price": 24.9
  },
  {
    "purchase_order": "DL-2026-0099",
    "created_at": "2026-09-15",
    "line": 10,
    "material": "EMB-500",
    "description": "Caixa papelão 40x30",
    "uom": "UN",
    "quantity_ordered": 100,
    "quantity_received": 0,
    "unit_price": 3.75
  }
]
}

```

Particularidades do Delta:

- Os nomes e os formatos dos campos são os mesmos do Alfa, inclusive o vocabulário de situação (**open**, **closed**, **blocked**). O que muda é a estrutura da entrega, não o formato dos dados.
- Montar o pedido completo é trabalho do consumidor: é você que junta cada item ao seu cabeçalho pelo **purchase_order**.
- Cada item traz também o seu próprio **created_at**: é a data em que aquela linha foi criada no pedido, que pode ser posterior à data do cabeçalho quando o item foi incluído depois.
- As duas consultas são independentes e **não há garantia de que retratem o mesmo instante**: uma pode conhecer um pedido que a outra ainda não conhece. Decida o que o seu serviço faz quando os dois lados não se encontram, registre no README por que decidiu assim e esteja pronto para defender.

Integre **os dois clientes** ao seu serviço. Os três requisitos da plataforma continuam valendo, agora para os quatro clientes.

No README, conte **o que você precisou mudar** para cada um deles entrar: o que foi só adicionar e o que exigiu mexer no que já existia — incluindo o que aconteceu com o banco (precisou de migração? o modelo de dados aguentou?).